



جامعة خليفة
Khalifa University

Enhancing Microservice Prototyping Platform for Virtual Reality Workloads: Adapting and Deploying the IR Engine

by

Ahmed Salem Alkaabi

MSc Thesis in Computational Data Science

December 2025

A thesis submitted to Khalifa University of Science and Technology in accordance with the requirements of the degree of MSc in Computational Data Science in the Department of Computer Science.

Enhancing Microservice Prototyping Platform for Virtual Reality Workloads: Adapting and Deploying the IR Engine

by

Ahmed Salem Alkaabi

A thesis submitted in partial fulfillment of the
requirements for the degree of

MSc in Computational Data Science

at

Khalifa University

Thesis Committee

Dr. Ibrahim Elfadel (Main
Advisor),
*Computer and Information
Engineering*
Dr. Lamees Alqassem (Co-
Advisor)
Computer Science

Dr. Chan Yeun (RSC Member
1),
Computer Science
Dr. U Zeyar Aung (RSC
Member 2)
Computer Science

December 2025

Abstract

Ahmed Salem Alkaabi, “**Enhancing Microservice Prototyping Platform for Virtual Reality Workloads: Adapting and Deploying the IR Engine**”, MSc Thesis in Computational Data Science, Department of Computer Science, Khalifa University of Science and Technology, United Arab Emirates, December 2025.

With the increasing complexity of virtual reality applications, scalable platforms capable of handling real-time user interactions and virtual environments have become crucial. Deploying virtual reality workloads as microservices is motivated by the need for flexible, scalable, and resource-efficient systems capable of handling the high demand for real-time interactions and 3D rendering in virtual environments. Traditional monolithic architectures often face bottlenecks in scaling and resource optimization, making them less suitable for the dynamic needs of virtual reality platforms. This research focuses on overcoming these limitations by containerizing services and deploying them on Docker and Kubernetes, enabling better resource management and improved system stability.

I surveyed frameworks for virtual reality workloads and analyzed the microservices frameworks for different workloads in the literature. After the analysis, I integrated the IR Engine (formerly called Ethereum Engine), an open-source virtual reality platform, into a microservice prototyping architecture to support virtual reality workloads in this Research. The focus is on addressing challenges related to resource allocation and utilization, scaling, dependency management, service orchestration, and real-time monitoring. The services are to be containerized as microservices and deployed on cloud infrastructure using Docker and Docker-Compose. The microservices are further deployed, managed, and orchestrated efficiently on a Kubernetes cluster. Additionally, industry-adopted tools like Grafana, Prometheus, Elasticsearch, and Kibana will also be deployed as microservices to enhance Log analysis, performance visualization, and monitoring capabilities. The findings from this research will provide practical solutions for deploying and optimizing virtual reality workloads in a microservices architecture. The goal is to provide insights into the challenges and solutions for deploying a reliable and scalable microservice-based virtual reality platform on the cloud, capable of supporting large-scale virtual reality applications.

Indexing Terms: Virtual reality, Microservices Architecture, Kubernetes, Scalability, IR Engine (Ethereal Engine).

Acknowledgement

I would like to express my deepest gratitude to my supervisors and advisors for their continuous guidance, patience, and encouragement throughout this research. Their insights and constructive feedback were invaluable in shaping this work into its final form.

I am also grateful to the faculty and staff of the Computer Science department at Khalifa University for providing the resources and an excellent academic environment that enabled me to pursue this project.

Special thanks go to my colleagues and friends who supported me, shared ideas, and provided motivation during the challenging stages of this research.

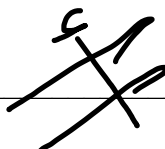
Finally, I owe my heartfelt gratitude to my family for their unconditional love, understanding, and constant encouragement. Their support enabled me to remain committed and complete this thesis.

Declaration and Copyright

Declaration

I declare that the work in this thesis was carried out in accordance with the regulations of Khalifa University of Science and Technology. The work is entirely my own except where indicated by special reference in the text. Any views expressed in the thesis are those of the author and in no way represent those of Khalifa University of Science and Technology. No part of the thesis has been presented to any other university for any degree.

Author Name: Ahmed Salem Alkaabi

Author Signature:  _____

Date: December 7, 2025

Copyright ©

No part of this thesis may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, without prior written permission of the author. The thesis may be made available for consultation in Khalifa University of Science and Technology Library and for inter-library lending for use in another library and may be copied in full or in part for any bona fide library or research worker, on the understanding that users are made aware of their obligations under copyright, i.e. that no quotation and no information derived from it may be published without the author's prior consent.

Table of Contents

Abstract	I
Acknowledgement	III
Declaration and Copyright	IV
List of Figures	VII
List of Tables	VII
List of Abbreviations	VII
1. Introduction	1
1.1 Motivation and Problem Description	1
1.2 Main Research Objectives	1
1.3 Research Questions	1
1.4 Research Gap and Contributions	2
2. Background	4
2.1 The Virtual Reality: Concepts and Requirements	4
2.2 Monolithic Architecture	4
2.3 Microservices Architecture	5
2.4 IR Engine Overview	6
2.4.1 Why the IR Engine Was Selected for This Research:	7
2.5 Key Tools and Technologies	9
3. Literature Review	12
3.1 Related Work and Case Studies	12
Case Study 1: Performance Impact of Microservices Architecture (Ramu, 2023)	12
Case Study 2: Evaluating the Impact of Cloud-Based Microservices on Application Performance (Desina, 2023)	12
Case Study 3: <i>"Lifting the veil on Meta's microservice architecture: Analyses of topology and request workflows"</i> (Huye, et al.)	13
Case Study 4: <i>"Research Opportunities in Microservices Quality Assessment: A Systematic Literature Review"</i> (Tapia, et al.)	13
Case Study 5: <i>"Netflix's Cloud Migration and Microservices Transformation"</i> (Netflix Blog)	13
Case Study 6: <i>"Microservices at Amazon"</i> (DreamFactory Blog)	14
Case Study 7: <i>"Microservices at eBay"</i> (Sayonetech Blog)	14
4. Methodology	14

5. IR Engine Microservices Architecture and Implementation	15
5.1 Monolithic to Microservices Conversion Process	15
5.1.1 IR Engine Monolithic Architecture Overview	15
5.1.2 Decomposition Strategy and Methodology	16
5.1.3 IR Engine Microservices Implementation with Helm Charts (MicroK8s Deployment)	17
5.2 Code Adaptation and Integration	22
5.3 Containerization and Orchestration	23
5.4 Deployment	24
5.5 Kubernetes Orchestration for IR Engine Microservices	26
5.5.1 Kubernetes Deployment Architecture (MicroK8s)	26
5.5.2 Service Discovery and Load Balancing	28
5.5.3 Scaling Policies	28
5.5.4 Docker-Compose vs. Kubernetes Usage	28
5.6 Monitoring and Testing	29
5.6.1 Resource Utilization and Performance Monitoring Approaches	30
5.6.2 Monitoring Architecture and Stack	30
5.6.3 Microservices Setup (Kubernetes/MicroK8s)	31
Grafana Dashboard from Microservices Deployment	32
6. Experimental Setup and Performance Analysis	32
6.1 Experimental Environment and Configuration	33
6.1.1. Hardware and Infrastructure	33
6.1.2 Software Stack	33
6.1.3 Test Data and Workload Generation	33
6.2 Performance Metrics and Measurement Methodology	34
6.2.1 Key Performance Indicators (KPIs)	34
6.2.2 Data Collection Methodology	35
6.3 Monolithic vs. Microservices Performance Comparison	36
6.4 Autoscaling and Scalability Behavior	39
6.5 Justification and Specific Bottlenecks Addressed	40
7. Discussion	42
8. Summary of Findings	43
9. Conclusion and Future Work	44
References/Bibliography	45

List of Figures

1. Monolithic vs Microservices Architecture (Section 2.3)
2. Monolithic Architecture of IR-Engine (Section 5.1.1)
3. IR Engine Microservice Architecture Diagram (Section 5.1.3)
4. UML Diagram for IR Engine Platform (Section 5.4)
5. Monitoring Setup Using Prometheus, Grafana, and ElasticSearch/Kibana (Section 5.6)

List of Tables

1. Comparison of IR Engine with other platforms/engines (Section 2.4.1)
2. CPU Utilization Comparison (%)
3. System Load Comparison (%)
4. Memory Usage Comparison (GB)
5. Response Time (p95) Comparison (ms)
6. Throughput Comparison (RPS)
7. Error Rate Comparison (%)

List of Abbreviations

- IR: Infinite Reality
- VM: Virtual Machine
- DB: Database
- CI/CD: Continuous Integration/Continuous Deployment
- K8s: Kubernetes
- R&D: Research and Development
- VUs: Virtual Users

1. Introduction

1.1 Motivation and Problem Description

The need for scalable platforms to efficiently manage virtual reality workloads is increasing rapidly. Traditional monolithic architectures struggle to handle the real-time demands of virtual environments, resulting in bottlenecks and inefficient resource usage. Microservice architectures, with their modularity and scalability, provide a more efficient solution. However, deploying virtual reality workloads within these architectures presents challenges, including dependency management, container orchestration, and the maintenance of real-time performance. This research aims to address these challenges by adapting and deploying the IR Engine [1] as a microservice-based platform for virtual reality workload deployment.

1.2 Main Research Objectives

- Transition and adapt the virtual reality Platform, i.e., IR Engine (formerly called Ethereal Engine) platform to a microservices-based architecture to improve resource efficiency, scalability, and performance.
- Overcome the technical challenges of dependency management, microservices cloud deployments, and service orchestration.
- Deploy the platform on cloud infrastructure using microservices and container management/orchestration tools with Automation.
- Implement advanced monitoring and observability using cutting-edge tools to track system performance, network throughput, and resource utilization.
- Reflect on case studies and other research papers to compare methodologies and outcomes.

1.3 Research Questions

- How can a microservices architecture improve the performance and scalability of virtual reality platforms such as IR Engine?
- What are the main challenges in adapting the IR Engine to a microservices architecture?
- How do microservices contribute to enhanced fault tolerance and resilience in virtual reality platforms compared to monolithic architectures?
- How can performance optimization be achieved through advanced monitoring tools like Grafana, Prometheus, and ElasticSearch?
- What are the key benefits of using cloud-native technologies (e.g., Docker, Kubernetes) for deploying and scaling virtual reality workloads?

1.4 Research Gap and Contributions

Recent work has explored microservice adoption and distributed execution models for virtual reality (VR) and extended reality (XR) environments. Examples include dynamic allocation of VR microservices to improve Quality of Experience (QoE) in distributed fog architectures [26], adaptive orchestration policies that balance cloud-edge resources for multi-user VR deployments [27], and remote-rendered VR streaming frameworks supporting heterogeneous devices [28]. Additionally, WebXR-based metaverse efforts propose browser-accessible virtual environments using A-Frame and related web components, emphasizing interoperability and client-side accessibility [29].

While these directions highlight the relevance of microservices and distributed execution for VR workloads, they do not directly align with the objectives of this thesis. Specifically:

Prior work is primarily simulation-oriented or evaluated on limited custom prototypes rather than on a fully featured, open-source metaverse engine with persistent services, real-time media, user sessions, and stateful components.

Existing approaches investigate allocation logic, latency balancing, or streaming pipelines but do not provide an end-to-end cluster-level evaluation of a real metaverse engine operating under load within a Kubernetes-orchestrated microservices architecture.

WebXR-based platforms report application-level accessibility metrics and rendering behavior but do not examine underlying infrastructure behavior, service bottlenecks, signaling load, or database/resource saturation in a containerized VR platform [29].

Documentation for iR Engine primarily emphasizes deployability and functionality but does not provide a reproducible study of iR Engine's scalability and performance as a microservices workload under VR-specific usage patterns. Accordingly, there is a gap between theoretical or streaming-focused VR orchestration frameworks and empirical characterization of a metaverse engine operating as a microservices platform under realistic VR load conditions.

This thesis addresses that gap through the following contributions:

Microservice-oriented deployment architecture for iR Engine on Kubernetes.

The platform decomposes core engine functions into service units and maps them to Kubernetes, including front-end, signaling/media, database, and supporting services, enabling controlled scaling and resource isolation.

Integration of platform-wide observability.

Metrics and logging components are incorporated to monitor compute, memory, and network usage per service and per node. The observability layer supports VR workload analysis across application and infrastructure boundaries.

Definition of a reproducible VR workload methodology.

The methodology specifies user concurrency levels, session distribution, and interaction patterns to evaluate platform response under increasing VR load, including measurement intervals and criteria.

Empirical characterization of performance and saturation behavior.

Experiments identify early bottlenecks, resource growth trends, signaling pressure, and the impact of

reservation and limit configurations. Findings reflect full-system behavior under load rather than theoretical allocations.

Deployment and tuning guidance for open-source metaverse engines.

Results are consolidated into practical recommendations concerning cluster sizing, reservation ranges, monitoring requirements, and service behavior expectations when hosting VR environments at scale.

Grounded in an actual engine deployment rather than simulated allocation constructs or isolated WebXR demonstrations, the presented work provides a measured platform-level evaluation of a metaverse engine operating as a Kubernetes microservices workload under VR conditions.

This research also contributes to the following areas:

A scalable, microservice-based platform designed to handle virtual reality workloads.

Solutions to resource allocation inefficiencies and scalability challenges typically encountered in monolithic application deployments, which will be analyzed and discussed in detail with performance results later in this work. A solution to the common dependency and orchestration challenges in real-time virtual reality applications.

A real-time monitoring and performance optimization framework using Prometheus [6], Grafana [5], Elasticsearch [7], and Kibana [8].

Offering practical insights into deploying virtual reality applications, while highlighting the need for future comparisons with established virtual reality benchmarks or literature-reported applications.

Thesis Outline

This thesis begins with the introduction, which presents the research motivation and objectives. The background and literature review section explores related work and relevant technologies. The methodology then describes the approach for adapting and deploying the IR Engine microservices architecture. This is followed by a detailed explanation of the design and implementation of the IR Engine microservices. The experimental setup and performance analysis section outlines the benchmarks and measurements conducted, while the discussion section interprets these results. The summary of findings highlights the key outcomes of the work. Finally, the conclusion and future work section reflects on the study and proposes next steps. Additional supporting material, including the expected outcomes and a high-level work plan, is provided at the end, along with references.

The relevant work from previous studies is presented in Section 3.1, “Related Work and Case Studies”.

2. Background

2.1 The Virtual Reality: Concepts and Requirements

The concept of virtual reality marks a transformative evolution in the way humans interact with digital environments. At its core, virtual reality is envisioned as a persistent, shared, and immersive virtual space that seamlessly integrates aspects of both physical and digital realities. It is not just a single application or platform but rather a complex, interconnected network of virtual worlds where users can socialize, work, play, and create in real time.

Several defining features set virtual reality apart from traditional digital experiences. **Persistence** refers to the idea that the virtual environment continues to exist and evolve, even when individual users log off. **Synchronicity** ensures that users across the globe can interact with each other and with the environment in real time. **Unlimited concurrency** implies the ability to support potentially millions of users simultaneously, without degradation in performance or experience. Lastly, **interoperability** is critical, as it allows users to carry their identities, assets, and experiences seamlessly across various platforms and virtual environments.

Meeting these ambitious goals presents considerable technical challenges. For virtual reality to function effectively, it requires a robust and highly responsive infrastructure. This includes systems that are **highly scalable**, capable of handling sudden spikes in user demand; **fault-tolerant**, ensuring continuous operation even when individual components fail; and **low-latency**, delivering near-instantaneous interactions to users regardless of their geographic location.

The architectural decisions behind building a virtual reality play a central role in addressing these challenges. Choosing the right architecture, whether centralized, decentralized, or a hybrid approach, directly influences performance, scalability, and user experience. For example, distributed systems may enhance scalability and fault tolerance but also introduce complexities around data consistency and coordination.

2.2 Monolithic Architecture

Monolithic architectures, where all application components are tightly integrated into a single deployable unit, have been the traditional approach for many applications. However, for large-scale, complex systems like virtual reality platforms, monoliths present several drawbacks:

- **Scalability Limitations:** Scaling requires replicating the entire application, which is inefficient if only specific components are bottlenecks.
- **Technology Lock-in:** The entire application is typically built using a single technology stack, hindering the adoption of optimal tools for specific tasks.

- **Deployment Complexity:** Updating any part requires redeploying the whole application, increasing risk and downtime.
- **Limited Resilience:** A failure in one component can potentially bring down the entire system.

2.3 Microservices Architecture

Microservices architecture represents a significant shift in how modern software applications are designed and built. Unlike traditional monolithic systems, where all components are tightly integrated and operate as a single, indivisible unit, microservices break an application down into smaller, self-contained services. Each of these services focuses on a specific business capability and operates independently, even though they are part of the same overall system.

In practice, this means that instead of having one large codebase where changes in one part might unexpectedly affect others, developers work with multiple smaller codebases. These microservices communicate with one another using lightweight communication protocols, most commonly RESTful APIs over HTTP/HTTPS or messaging systems such as RabbitMQ or Apache Kafka. This loose coupling between services is key, it ensures that teams can develop, test, deploy, and scale different parts of the application independently, without disrupting the entire system.

This architectural style offers several clear advantages. First, it significantly accelerates development cycles. Since each microservice can be developed and deployed separately, teams can work in parallel, delivering new features and updates more quickly. Second, microservices enhance scalability. Because each service can be scaled on its own, resources can be allocated more efficiently based on the specific performance needs of individual components. This is especially valuable in cloud-native environments where infrastructure can be elastically provisioned. Third, the modular nature of microservices supports better fault isolation. If one service fails, it doesn't necessarily bring down the entire application; only the functionality provided by that service is affected.

On the contrary, monolithic architectures present several challenges. As all application components are bundled together, even minor changes often require a full rebuild and redeployment of the entire system. This tightly coupled structure makes it harder to isolate faults, slows down the development process, and limits the ability to scale components independently. As the application grows, so does the complexity, which can quickly become a bottleneck for innovation and maintenance.

Figure 1 shows a high-level comparison between monolithic and microservices architectures, highlighting how responsibilities and functions are distributed in each model.

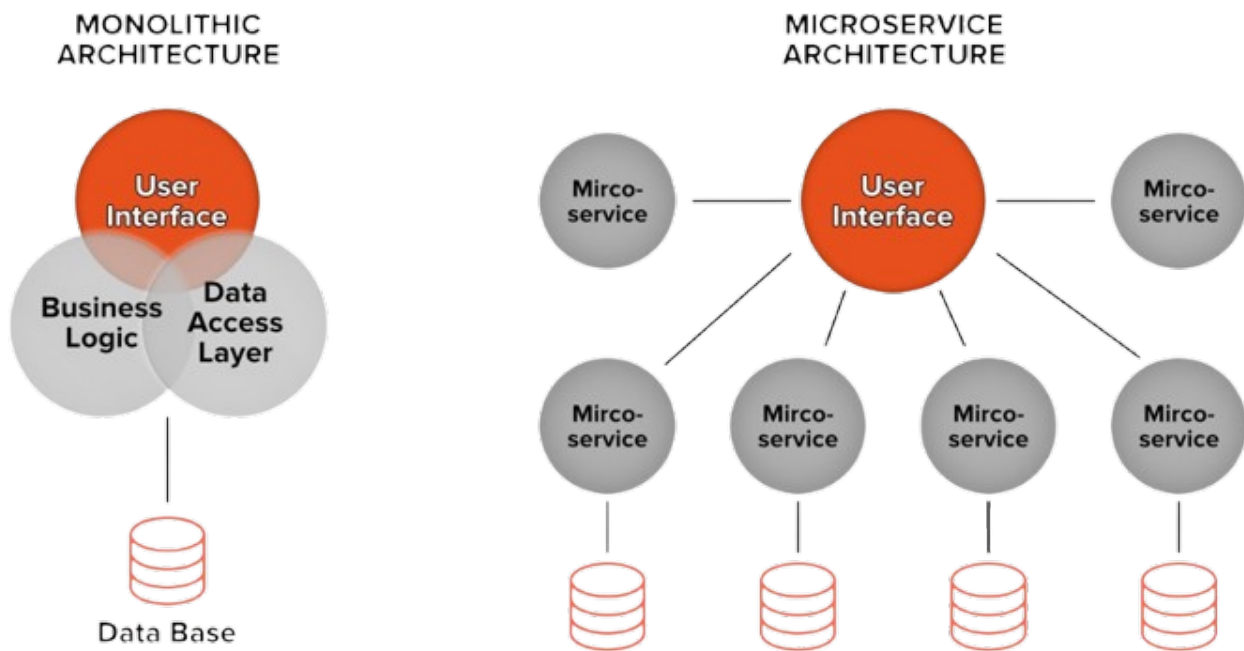


Figure 1: Monolithic vs Microservices Architecture

Microservices typically rely on:

- **APIs:** To communicate between the services.
- **Containerization:** Tools like Docker encapsulate each service with its runtime environment and dependencies, which ensures consistent deployment across development, testing, and production environments.
- **Management and Orchestration:** Docker-Compose is used to run multi-container Docker applications. It allows the management and orchestration of multiple containers that make up a single application. Kubernetes, on the other hand, automates deployment, scaling, load balancing, and failover/auto-healing for containers, making microservices architectures manageable at scale.

2.4 IR Engine Overview

The IR Engine, formerly known as Ethereum Engine, is a Free, open-source, & Interoperable Immersive Web Technology platform designed to support virtual reality applications. It is built for the web, Its functionalities include real-time communication via WebRTC [15] (an open-source project providing web and mobile applications with Real-Time Communication), 3D environment rendering, and support for multiplayer interactions. Compared to platforms like Unity and Unreal Engine, IR Engine

emphasizes ease of deployment and open-source collaboration. Unlike proprietary engines, it offers developers the flexibility to customize and scale their deployments without licensing constraints.

2.4.1 Why the IR Engine Was Selected for This Research:

1. **Open-Source Nature:**

The IR Engine is an open-source platform, making it flexible and accessible for academic research and development purposes. Its open codebase allows researchers to analyze, adapt, and optimize its functionalities without being restricted by proprietary limitations.

2. **Specialized for virtual reality Applications:**

IR Engine is explicitly designed for virtual reality workloads, handling real-time 3D interactions, immersive environments, and large-scale user interactions. These are critical components for the research, which aims to optimize virtual reality platform performance through microservices.

3. **Support for Real-Time Communication:**

The platform incorporates WebRTC (Web Real-Time Communication), allowing low-latency communication crucial for real-time user interactions in virtual environments. This makes it a good choice for experimenting with scalable and responsive systems.

4. **Existing Microservice-Ready Architecture:**

The IR Engine already provides containerized components through publicly available Docker images and Helm charts, enabling flexible deployment on Kubernetes platforms such as microk8s. However, while these images simplify the modular deployment of supporting services like databases and Redis, the IR Engine as a whole was originally designed to run in a more monolithic style when deployed without orchestration platforms such as Kubernetes. For this research, these supporting services (databases, Redis, MinIO) will also be deliberately deployed in a monolithic, non-containerized form to better measure and compare the transition toward a fully microservice-based deployment. Therefore, the IR Engine will be first deployed in its monolithic form to establish baseline benchmarks. Subsequently, it will be adapted to a microservices-based deployment on microk8s, leveraging its Helm charts, Kubernetes manifests, and custom observability enhancements to fully reveal the benefits of microservice-based scaling, orchestration, and monitoring. This approach ensures that the platform can be evaluated under both monolithic and microservices architectures, fulfilling the research objective of studying the transition of virtual reality workloads to microservices.

5. **Cross-Platform and Browser Accessibility:**

IR Engine enables users to access immersive experiences directly via a browser without needing to download standalone clients. This lightweight deployment model complements cloud-native strategies and improves accessibility for end users.

6. **Ease of Observability:**

The platform integrates seamlessly with monitoring tools such as Elasticsearch and Kibana,

making it easier to track performance and metrics as the services are containerized and deployed.

Feature	IR Engine	Unity	Unreal Engine	NVIDIA Omniverse
Licensing	Open-source	Proprietary	Proprietary	Proprietary
Browser-based	Yes	No	No	No
Scalability	High (Kubernetes)	Moderate	High	High
Real-Time Features	Yes (WebRTC)	Yes	Yes	Yes

Table 1: Comparison of the IR Engine with other platforms/engines

Key challenges in this research involved resolving dependency and deployment issues with components such as Node.js [9], Mediasoup [10], and configuring Docker and Kubernetes. The research aimed to decouple monolithic services into independent microservices to achieve better resource optimization, scalability, and overall performance, particularly for high-demand environments like the virtual reality. The open-source IR Engine platform proved well-suited for this transformation, requiring some development adjustments and configuration changes to support containerization of its core components. These adjustments enabled the services to be separated into microservices, which were then deployed using Docker and Docker Compose. Finally, the platform was orchestrated on Kubernetes (MicroK8s), enabling robust deployment and management. Extensive benchmarking and resource utilization measurements were carried out to compare the microservices-based deployment against the traditional monolithic setup, quantifying improvements in both efficiency and scalability.

A critical aspect of the research is to study how this architecture improves performance and scalability in cloud environments, especially for virtual reality workloads, in comparison to traditional monolithic deployments. The primary advantage of microservices architecture lies in its ability to handle large-scale, real-time applications (such as those seen in the virtual reality) by distributing workloads efficiently across different components, reducing bottlenecks.

2.5 Key Tools and Technologies

- **Docker**

Docker is an open-source platform that helps developers build, deploy, and manage applications inside containers. Containers are lightweight, standalone, and executable units of software that include everything needed to run an application, such as the code, runtime, system libraries, and dependencies. This makes applications portable and ensures consistency across environments.

Key Features of Docker:

1. **Portability:** Containers work the same way across different operating systems and infrastructures. Whether on a developer's local machine, a test server, or a production cloud environment, Docker ensures consistent behavior.
2. **Efficiency:** Docker containers share the host machine's OS kernel, reducing overhead compared to virtual machines.
3. **Isolation:** Each container runs independently, preventing conflicts between applications or dependencies.
4. **Version Control:** Docker allows version containers, making it easy to roll back to a previous version if needed.

In this research, Docker is used to containerize the different components (e.g., frontend and backend server, Redis cache, MariaDB databases, monitoring tools like CAdvisor, Prometheus and Grafana, Elasticsearch and Kibana, etc.) of the IR Engine. Each container runs a microservice, providing isolated environments that enable seamless deployment, scalability, and flexibility. Containers ensure consistency across development and production environments.

- **Docker-Compose**

Docker-Compose is a tool for defining and managing multi-container Docker applications. A simple YAML file allows developers to describe how different services in an application interact and ensures they start in the correct order.

Key Features of Docker-Compose:

1. **Multi-Container Orchestration:** It helps manage applications with several interconnected services, like a database, backend API, and front-end client.
2. **Simplicity:** A single docker-compose.yml file can define all services, making the setup straightforward.
3. **Environment Management:** It allows the use of environment variables, making the configuration flexible for different deployment setups.
4. **Networking:** Docker-Compose automatically sets up a shared network for all services in the stack.

Within the IR Engine, Docker Compose simplifies running containers that involve complex configurations or dependencies across multiple microservices.

- **Kubernetes**

Kubernetes (aka K8s) is an open-source platform for managing containerized applications across a cluster of machines. It automates deployment, scaling, and operations, making it best for large-scale, production-grade applications.

Key Features of Kubernetes:

- **Container Orchestration:** Kubernetes manages the lifecycle of containers, ensuring they run as intended and expected.
- **Scaling:** Automatically adjusts the number of running containers based on demand, making it highly efficient during traffic spikes.
- **Service Discovery and Load Balancing:** Exposes services to external clients and distributes traffic evenly across containers.
- **Self-Healing:** If a container crashes, Kubernetes automatically restarts it to maintain application uptime.
- **Storage Orchestration:** Manages storage needs dynamically, including persistent volumes for databases.

Kubernetes handles the orchestration of containers in the platform. It automates container deployment, scaling, and management, allowing IR Engine to manage a higher number of containers efficiently. The system is used to maintain service availability, auto-scaling, and load balancing, and ensure minimal downtime in the system's operation.

We'll use **MicroK8s** [23], which is the Kubernetes distribution provided by Canonical and is primarily suitable for installing or deploying a single-node Kubernetes cluster.

- **Helm:**

Helm [24] is a package manager for Kubernetes, used to manage and deploy applications in containerized environments. It simplifies the process of deploying software to Kubernetes

clusters by packaging the application, its dependencies, and configurations into a single unit called a chart.

- **Redis**

Redis serves as an in-memory data structure store and distributed caching solution. Positioned between the backend server and databases (MariaDB in this case), it caches frequently accessed data to reduce latency and improve data retrieval speed. For IR Engine, Redis helps improve the overall system's performance, particularly for real-time virtual reality workloads.

- **MariaDB**

The IR Engine uses multiple instances of **MariaDB**, a high-performance relational database, for storage. Each microservice connects to its specific database instance, further improving performance and isolating failures. It is used as the default database for the IR Engine platform, possibly due to its high-performance capabilities, open-source nature, and compatibility with microservices architecture.

ir_engine-db, **testdb**, **minikubedb** are multiple instances of MariaDB, handling different datasets and workloads.

- **Agones**

Agones is used as the instance server for managing and scaling dedicated game servers, which is critical for virtual reality-like real-time applications that require low latency and high availability performance.

- **Prometheus & Grafana**

Prometheus is used for monitoring the health and performance of the platform by collecting time-series data from various services, while **Grafana** visualizes these metrics, offering insights into resource usage, latency, and potential bottlenecks. This combination is important for analyzing the performance improvements and optimized resource utilization of the microservices architecture over the monolithic setup.

- **CAdvisor**

CAdvisor monitors resource usage (CPU, memory, disk, network) by containers, making it a key tool for understanding the performance characteristics of each microservice within the IR Engine.

- **Elasticsearch and Kibana**

Elasticsearch serves as the data search and analysis engine, while Kibana visualizes and queries the data. In IR Engine, these tools help aggregate and analyze logs from microservices, enabling easier troubleshooting, monitoring, and performance analysis.

- **K6**

K6 is a modern load-testing tool designed for developers and testers. It supports tests as code, configurable load generation, and a rich API. With support for multiple protocols, k6 is suitable for both monolithic applications and microservices in load testing and performance analysis scenarios.

3. Literature Review

3.1 Related Work and Case Studies

Several studies have explored the benefits of microservices architecture over monolithic systems, particularly in environments requiring high scalability and real-time processing, such as gaming and virtual reality platforms. Microservices allow for rapid scaling, easier fault isolation, and more effective resource allocation.

The shift toward containerization using Docker and orchestration with Kubernetes has become critical for handling virtual reality-like workloads, where user interaction is dynamic and resource demands fluctuate rapidly. Redis, as a distributed cache, has been pivotal in reducing latency in systems with high user demands.

Case Study 1: Performance Impact of Microservices Architecture (Ramu, 2023)

This study highlights how microservices can enhance scalability and resource efficiency in cloud-based environments. By breaking down applications into smaller, independently deployable services, microservices allow for more granular control over resources. This is particularly beneficial for virtual reality workloads, which often require dynamic scaling to handle fluctuating user demands. However, one challenge identified is the increase in communication overhead between services. To address this, the study recommends using more efficient communication protocols, such as gRPC, which can reduce the latency typically experienced in distributed systems. Furthermore, the study emphasizes the resilience of microservices, as service failures are isolated, ensuring that the overall system remains stable, a crucial feature for maintaining real-time virtual reality interactions.

Case Study 2: Evaluating the Impact of Cloud-Based Microservices on Application Performance (Desina, 2023)

This study explores the performance benefits of adopting cloud-based microservices architectures, especially in terms of response time and system throughput.

For virtual reality platforms, which rely heavily on real-time interactions, the ability to independently scale and optimize services is important. The study highlights the advantages of containerization technologies like Docker and Kubernetes in achieving flexible deployment and auto-scaling

capabilities. Additionally, the research underscores the importance of monitoring and troubleshooting in a distributed microservices environment. Tools like Prometheus and Grafana are recommended for tracking real-time metrics such as CPU usage, memory, and service health, which are critical for ensuring the system's reliability and performance.

Case Study 3: *"Lifting the veil on Meta's microservice architecture: Analyses of topology and request workflows"* (Huye, et al.)

This study provides an in-depth examination of Meta's (formerly Facebook) microservices architecture. It highlights the need for large-scale distributed systems, particularly for handling vast amounts of traffic from virtual reality-related workloads. Meta's platform leverages microservices to enhance scalability, fault tolerance, and resource efficiency. By examining the internal architecture, the thesis highlights how Meta has successfully utilised a microservice architecture to manage communication between services via load balancers and fault-tolerant APIs. The performance monitoring techniques utilized include latency and throughput analysis to optimize network bottlenecks. These methods are required when deploying virtual reality workloads, as such systems demand real-time interactions with minimal delays.

Key takeaway: Microservices enable enhanced scalability and reduced failure impact, which is especially beneficial when working with resource-intensive platforms such as those handling virtual reality workloads.

Case Study 4: *"Research Opportunities in Microservices Quality Assessment: A Systematic Literature Review"* (Tapia, et al.)

This thesis systematically reviews the current state of research related to microservices, focusing on the quality assessment criteria. It identifies that performance, security, scalability, and maintainability are key factors when evaluating microservice-based systems. The review highlights several tools that can be used for performance assessment, like Prometheus and Grafana, which are widely used to measure the performance of microservices in real-time. The study highlights the need for continuous testing and monitoring in distributed systems, which aligns with this research goal for deploying and comparing virtual reality workloads on cloud-based microservices architecture.

Key takeaway: The systematic review shows that evaluating the performance, maintainability, and scalability of microservices is necessary for successful deployment. Monitoring tools play a key role in performance tracking, which aligns with our research focus on virtual reality workload optimization.

Case Study 5: *"Netflix's Cloud Migration and Microservices Transformation"* ([Netflix Blog](#))

Netflix's migration to microservices was driven by scalability needs and the desire to move away from monolithic architectures. Like the virtual reality, Netflix's services require real-time interaction and the ability to scale globally. By breaking down its monolithic applications into microservices, Netflix could achieve rapid deployment cycles and elastic scalability. Their architecture uses Amazon Web Services (AWS) for global distribution, allowing seamless scaling and service uptime.

This type of architecture is highly suited for virtual reality workloads, which require similar real-time processing and elasticity.

Key takeaway: Netflix’s shift to microservices illustrates the effectiveness of microservice architectures for real-time applications requiring massive scalability, a critical element for deploying virtual reality workloads.

Case Study 6: "Microservices at Amazon" ([DreamFactory Blog](#))

Amazon’s adoption of microservices was a turning point in how they scaled their services to meet growing user demands. By transforming their monolithic system into microservices, Amazon could isolate features such as the “Buy” button or tax calculator into separate services. This granular approach allowed for independent scaling and quicker response times, critical for large-scale operations such as virtual reality platforms that demand fast processing and responsiveness. Their architecture also emphasizes elasticity and fault tolerance, characteristics that are highly relevant to our work on deploying virtual reality workloads as microservices.

Key takeaway: Amazon’s approach to microservices, which highlights independent scaling and resource optimization, closely aligns with the goals of optimizing virtual reality workloads for performance and resource utilization.

Case Study 7: "Microservices at eBay" ([Sayonetech Blog](#))

eBay’s journey towards microservices helped them resolve scaling and processing challenges associated with their monolithic architecture. By moving towards a microservices architecture with Docker and Kubernetes, eBay achieved agility and fault tolerance. The study highlights eBay's use of polyglot (multi-language) microservices and the circuit breaker pattern for fault isolation. These tools and patterns are essential for managing high-throughput services, which are necessary for efficient virtual reality workloads. eBay’s case reinforces the advantages of microservices in scaling and handling complex workloads, and their methods of deploying independent services can be directly applied to optimizing virtual reality workloads.

Key takeaway: eBay’s transition to microservices resulted in improved scalability, fault tolerance, and resource efficiency, which are the key elements for deploying virtual reality workloads using microservices.

4. Methodology

This research employs a comparative experimental methodology to evaluate the performance of monolithic versus microservices architectures for the IR Engine virtual reality platform. The methodology involves:

Architecture Implementation: Implementing both the original monolithic architecture and the

refactored the microservices architecture of the IR Engine.

Environment Setup: Configuring identical, controlled hardware and software environments for both architectures.

Workload Simulation: Designing and implementing realistic load testing scenarios using K6 to simulate concurrent user interactions typical of a virtual reality platform.

Monitoring and Data Collection: Setting up a comprehensive monitoring stack (Prometheus, Grafana) to collect key performance indicators (KPIs) directly from system dashboards.

Performance Evaluation: Executing load tests against both architectures and analyzing collected KPIs, including response time, throughput, resource utilization, error rates, and scalability.

Comparative Analysis: Systematically comparing the performance results of the two architectures to identify differences, advantages, disadvantages, and specific bottlenecks.

5. IR Engine Microservices Architecture and Implementation

5.1 Monolithic to Microservices Conversion Process

5.1.1 IR Engine Monolithic Architecture Overview

While the IR Engine offers container images to modularize supporting services like databases and Redis, the core platform was originally designed to operate in a monolithic style without orchestration tools such as Kubernetes. For this research, these supporting services (databases, Redis, MinIO) were also intentionally deployed in a monolithic, non-containerized manner to establish a baseline for comparing the fully microservice-based deployment.

This architecture consisted of several key functional modules operating within a single codebase and deployment unit:

- **Frontend Client:** A React-based web application handling user interface rendering and client-side logic
- **API Server:** A Node.js Express server managing all backend operations, authentication, and business logic
- **Real-time Communication:** WebRTC implementation via Mediasoup for voice, video, and data channels
- **World State Management:** Centralized logic for maintaining virtual environment state and synchronization

- **Database Layer:** Unified access to MariaDB for all data persistence needs
- **Asset Management:** Handling of 3D models, textures, and other media assets

This monolithic design presented several challenges as user demand increased:

- **Scaling Limitations:** The entire application had to be scaled as a unit, even when only specific components (like real-time communication) were under load
- **Resource Inefficiency:** Underutilized components consumed resources alongside heavily utilized ones
- **Development Bottlenecks:** The large codebase required extensive knowledge for developers and slowed down feature implementation
- **Deployment Complexity:** Updates required redeploying the entire application, increasing downtime risk
- **Fault Isolation:** Issues in one component could affect the entire system

Figure 2 illustrates the monolithic architecture of the IR Engine platform.

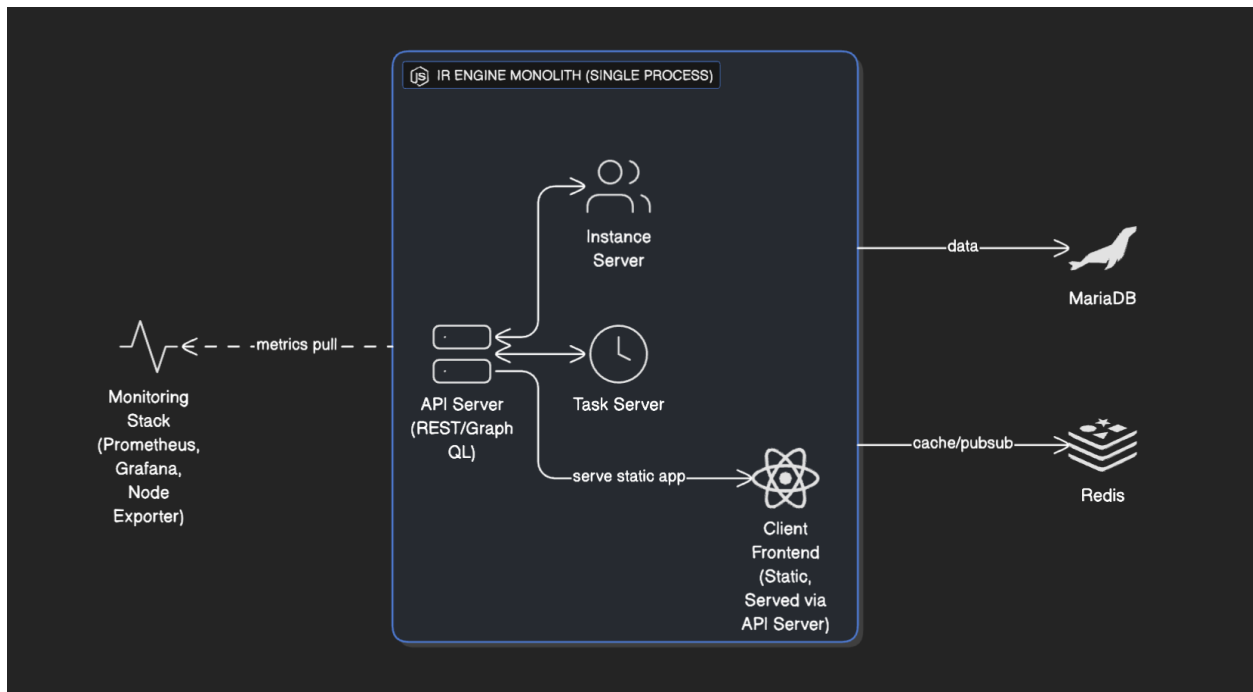


Figure 2 Monolithic Architecture of IR-Engine

5.1.2 Decomposition Strategy and Methodology

The transition to microservices followed a systematic approach, leveraging Helm charts and containerization. The bounded contexts within the monolith were identified based on their ability to function independently, and their extraction was prioritized based on:

- **Business Capability:** Each microservice encapsulates a specific business function
- **Data Cohesion:** Services own their data and minimize dependencies
- **Deployment Independence:** Services are deployable without affecting others
- **Scalability Requirements:** Components with different scaling needs were separated

The implementation process followed these methodical steps:

1. **Service Boundary Identification:** The IR Engine architecture was analyzed to identify natural service boundaries between components.
2. **Helm Chart Utilization:** Existing Helm charts were leveraged to define the deployment configuration for each microservice, including resource requirements and dependencies.
3. **Containerization:** Docker images were used for containerization, with service-specific dependencies and configurations maintained in separate containers.
4. **MicroK8s Deployment:** The single-node Kubernetes cluster (MicroK8s) was configured to deploy and manage the containerized services, providing a lightweight yet powerful orchestration environment.
5. **Pod Configuration:** Kubernetes pods were configured to host the containers, with appropriate resource limits and service discovery settings.
6. **Service Mesh Implementation:** The communication between services was managed through Kubernetes service abstractions, enabling discovery and load balancing.

5.1.3 IR Engine Microservices Implementation with Helm Charts (MicroK8s Deployment)

Microservices allow complex applications to be divided into independent services that communicate through APIs. This architecture enhances flexibility, enabling individual services to be updated, scaled, or restarted without disrupting the entire system. The IR Engine will be adapted to fit this architecture, making it more scalable and easier to manage. The following visualization provides an overview of the microservices approach (see Figure 3: Microservice Architecture - IR Engine Platform).

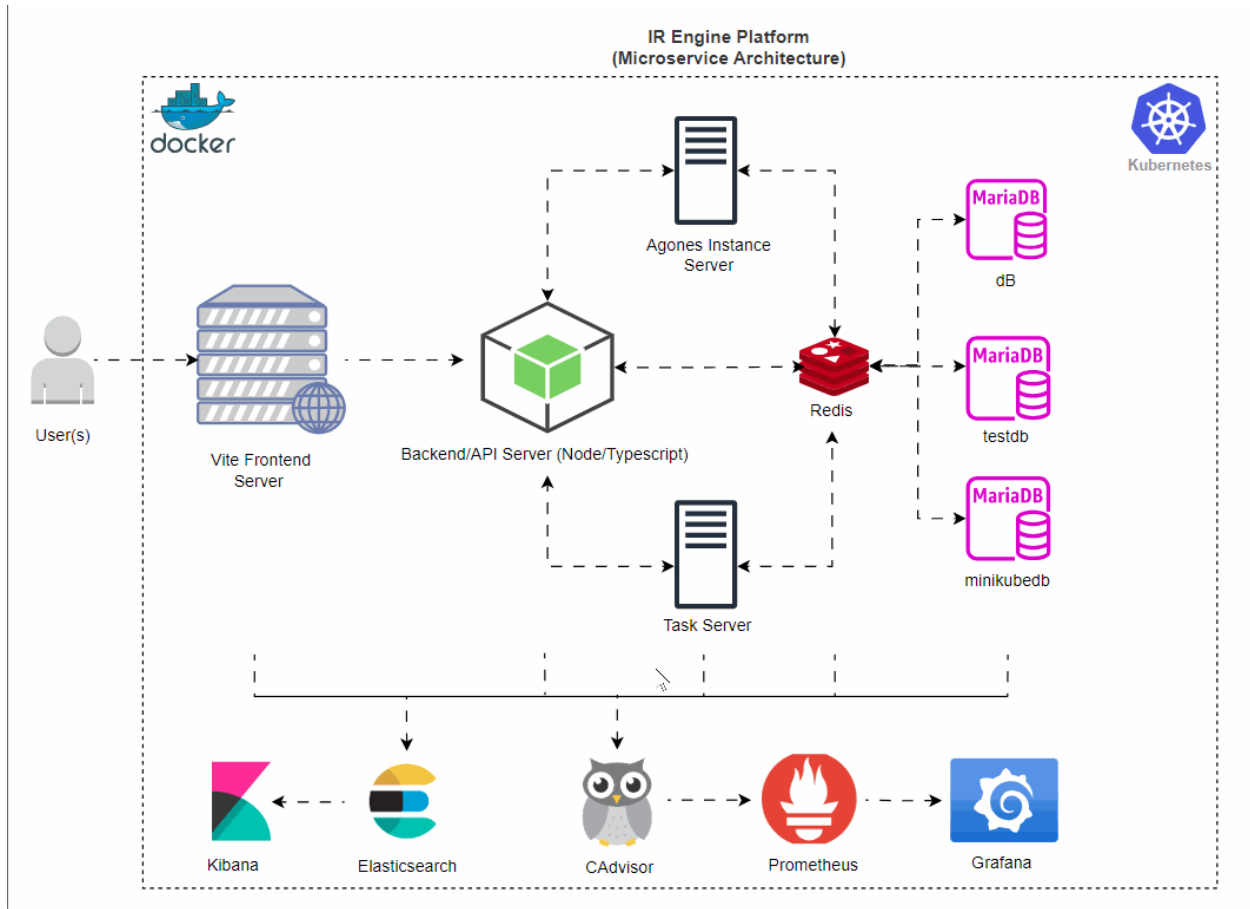


Figure 3: Microservice Architecture - IR Engine Platform

The diagram illustrates how these components of the IR Engine work together. Below is a brief overview of all these microservices:

1. Users will access the IR Engine's Vite [11] Frontend server from their browser using the correct address
2. The Backend Server i.e. NodeJS [9] acts as the central hub, communicating with various services through APIs and serving user requests.
3. Clients interact primarily with the Backend Server and Agones Instance Server for game-related operations.
4. The Backend Server i.e. NodeJS [9] queries and updates the primary MariaDB [12] database (db) for persistent data storage.
5. Redis [13] is utilized for caching frequently accessed data, reducing the load on the primary database.

6. Elasticsearch [7] powers search functionality and aggregate logs coming from the microservices, with Kibana [8] providing a centralized user interface for data/logs visualization and analysis.
7. The Taskserver interacts with the Backend Server and databases to perform scheduled and background operations.
8. CAdvisor [14] is used to get an understanding of the resource usage and performance characteristics of the running containers and it will push these metrics to Prometheus which is a time-series database for storing the metrics. The metrics can then be monitored on Grafana Dashboards.
9. Prometheus [6] and Grafana [5] provide advanced and real-time monitoring and performance visualization of the deployed microservices.

Helm Charts Overview

Helm is a package manager for Kubernetes that simplifies the deployment and management of applications by using a packaging format called charts. It automates the creation, configuration, and deployment of applications, making it easier to manage complex Kubernetes environments. A *Helm chart* is a collection of YAML templates and configuration files that define a set of Kubernetes resources as a single, versioned package.

In this research, Helm charts were used to deploy and manage the IR Engine's microservice ecosystem, providing a reproducible, configurable, and scalable installation of its various components.

The IR Engine platform's conversion from a monolithic to a microservices architecture was implemented using Helm charts for standardized deployment and configuration management on a MicroK8s cluster. This approach provided several advantages:

- **Templated Deployments:** Helm charts enabled parameterized deployments across different environments while maintaining consistent service configurations.
- **Dependency Management:** The hierarchical structure of Helm charts facilitated the management of dependencies between microservices and external components.
- **Versioned Configurations:** All configuration changes were versioned alongside the application code, ensuring reproducible deployments.

The implementation leveraged multiple Helm charts from public repositories and customized Helm values to orchestrate and manage the IR Engine ecosystem on Kubernetes. Each of these Helm charts provides a templated, reusable, and configurable approach to deploying complex application components. The following charts were integrated and configured as part of this research:

- **etherealengine chart (from <https://helm.etherealengine.org>):** This is the main chart that orchestrates the deployment of IR Engine’s critical services, including the API server, the front-end client, and the supporting file-server components. It allows parameters to be customized (e.g., ingress hosts, environment variables, port settings) to suit the target Kubernetes cluster and domain environment.
- **agones chart (from <https://agones.dev/chart/stable>):** Agones is an open-source, Kubernetes-native game server orchestrator. The Helm chart simplifies deploying Agones components, such as the controller and extension services, which then handle the lifecycle and allocation of multiplayer game server instances (the instance servers for IR Engine’s virtual reality scenes). In this research, Agones was essential to enable on-demand scaling and decommissioning (via Kubernetes) of instance servers depending on client load.
- **redis chart (from <https://charts.bitnami.com/bitnami>):** Redis serves as the in-memory caching and pub/sub messaging backbone for IR Engine, improving real-time communication, session persistence, and state distribution among services. The Redis Helm chart facilitates its deployment with configurable persistence, security, and high-availability options.
- **elasticsearch chart (from <https://helm.elastic.co>):** Elasticsearch acts as a powerful search and analytics engine, indexing logs and supporting queries for system events or user actions. Its Helm chart automates and deploys the Elasticsearch search and analytics engine.
- **kibana chart (from <https://helm.elastic.co>):** Kibana is a visualization and dashboard tool that integrates with Elasticsearch. Its chart sets up the Kibana interface in the cluster, including ingress rules and secure connections, so that system logs and performance metrics collected by Elasticsearch can be explored in a user-friendly manner.

The `microk8s.helm list` output confirms these deployments:

NAME	NAMESPACE	REVISION	UPDATED	STATUS
	CHART	APP VERSION		
agones	default	1	2025-03-18 20:16:32.169116231 +0000 UTC	deployed
	agones-1.47.0	1.47.0		
elasticsearch	default	1	2025-03-18 20:19:36.767557258 +0000 UTC	deployed
	elasticsearch-8.5.1	8.5.1		
kibana	default	1	2025-03-18 20:23:40.50831821 +0000 UTC	deployed
	kibana-8.5.1	8.5.1		
local	default	1	2025-03-18 21:59:26.106321063 +0000 UTC	deployed
	etherealengine-5.3.2	1.6.0		
local-redis	default	1	2025-03-18 20:17:03.14329373 +0000 UTC	deployed
	redis-20.11.3	7.4.2		

The `microk8s.kubectl get pods` output provides a clear view of the running microservice instances, demonstrating the granular decomposition and deployment:

NAME	READY	STATUS	RESTARTS	AGE
agones-controller-85ccd5594b-jmlv4	1/1	Running	0	15h
agones-controller-85ccd5594b-rxvrf	1/1	Running	0	15h
agones-extensions-574c6cfb45-r2gkq	1/1	Running	0	15h
agones-extensions-574c6cfb45-g8swg	1/1	Running	0	15h
local-etherealengine-client-5cd4fbd859-cxf6r	1/1	Running	0	15h
local-redis-master-0	1/1	Running	0	15h
local-redis-replicas-0	1/1	Running	0	15h
kibana-kibana-749869bc5c-wvdt2	1/1	Running	0	15h
local-redis-replicas-1	1/1	Running	0	15h
local-instanceserver-cpc66-vwbpr	2/2	Running	0	15h
local-instanceserver-cpc66-5zwqv	2/2	Running	0	15h
elasticsearch-master-0	1/1	Running	0	15h
elasticsearch-master-1	1/1	Running	0	15h
elasticsearch-master-2	1/1	Running	0	15h
local-etherealengine-api-c7fc9cdc5-kgdpd	1/1	Running	0	15h
local-redis-replicas-2	1/1	Running	0	15h

The resulting microservices architecture, as observed in the MicroK8s deployment, includes the following key components, each managed as independent pods:

Agones Controller & Extensions: These pods (`agones-controller`, `agones-extensions`) are part of the Agones game server orchestration system, responsible for managing and scaling dedicated game server instances for real-time virtual reality interactions.

IR Engine Client Service: The `local-etherealengine-client` pod serves the optimized frontend application to users' browsers.

IR Engine API Service: The `local-etherealengine-api` pod functions as the Backend Server, handling central API requests, communicating with other services, and interacting with the primary MariaDB database. This service likely encapsulates functionalities such as user authentication, profile management, virtual world definitions, and analytics processing, which were logically separated in the initial design.

IR Engine Instance Server: The `local-instanceserver` pods are the actual game server instances managed by Agones, crucial for real-time state synchronization and low-latency performance in the virtual reality.

Redis: The `local-redis-master` and `local-redis-replicas` pods provide an in-memory data store and distributed caching solution, reducing latency and improving data retrieval speed for frequently accessed data.

Elasticsearch: The `elasticsearch-master` pods serve as the data search and analysis engine, aggregating logs from all microservices.

Kibana: The `kibana-kibana` pod provides the centralized user interface for visualizing and querying the aggregated data from Elasticsearch, aiding in troubleshooting and performance analysis.

The Helm chart structure for the `ir-engine/etherealengine` application, which deploys the core IR Engine services, would typically follow a pattern similar to:

```
ir-engine/
├── Chart.yaml          # Chart metadata (name, version, dependencies)
├── README.md           # Deployment documentation
├── values.yaml         # Global and service-specific config
└── templates/
    ├── _helpers.tpl    # Template helpers/macros
    ├── api-server-*.yaml # API server
    ├── client.yaml      # Frontend client deployment
    ├── instanceserver.yaml # Real-time multiplayer instance server
    ├── media.yaml       # WebRTC media handler (Mediasoup)
    ├── taskserver.yaml  # Optional background task processor
    ├── batchinvalidator.yaml # Optional batch invalidation worker
    ├── ingress.yaml     # Ingress resources (NGINX)
    └── service.yaml     # Shared service template
```

This structure allows for independent scaling and configuration of each component while maintaining a unified deployment process through Helm. The `values.yaml` file would contain service-specific configurations, enabling flexible deployments across different environments.

5.2 Code Adaptation and Integration

The IR Engine supports both monolithic and microservices deployment models. In its monolithic form, the platform is designed as a semi-modular architecture, with tightly integrated services

launched as a unified process. This research leverages both deployment strategies to benchmark performance, scalability, and efficiency under controlled virtual reality workloads.

In the monolithic setup, the IR Engine components (such as API server, Instance Server, and Redis) were deployed natively on a physical/virtual machine using Node.js without Docker or Kubernetes. Services were initiated using npm scripts (e.g., `npm run start`, `npm run dev-reinit-db`), and database connections were configured with native MariaDB and Redis instances. Code-level adjustments included managing SSL support, disabling Kubernetes service detection where applicable, and setting environment flags to match the standalone configuration.

In the microservices setup, the same platform was deployed using Helm charts on a single-node MicroK8s Kubernetes cluster. Each component (API, client, Redis, Agones, Instanceserver) was containerized and isolated using standard Helm practices. Configurations were tuned to ensure parity in performance benchmarking, such as aligning CPU/memory resources across the two architectures.

5.3 Containerization and Orchestration

In the microservices model, containerization was a critical step. Each logical service of the IR Engine (API server, Instance Server, Redis, Agones Controller, etc.) was packaged into a separate Docker container.

- **Containerization:**

In this research, containerization was used to package each functional component of the IR Engine platform such as its API server, client interface, instance server, Redis data store, Agones game server controller etc. into standardized and portable containers. The official Docker images provided by Ethereum Engine for the API and Client were deployed through their maintained Helm charts, which simplify the container's configuration, networking, and service definitions within Kubernetes. It is important to note that the official IR containers function primarily as runnable artifacts, but not as a reproducible microservice platform. My contribution was in architecting a Kubernetes-controlled environment around them, defining independent service lifecycles, observability integration, resource policies, fault recovery, and workload measurement, allowing the VR stack to be evaluated, tuned, and scaled in ways not provided nor documented by the official deployment.

Redis and other supporting dependencies were containerized using popular Bitnami Helm charts, which provide best-practice configurations and ready-to-use Docker images. These Helm charts allowed the services to be deployed as Kubernetes pods with appropriate resource requests, limits, and persistent storage volumes, ensuring each container could operate independently while still being managed under Kubernetes orchestration.

- **Orchestration:**

To manage the deployment and scaling of the containerized IR Engine services, MicroK8s was selected as the Kubernetes orchestration platform. MicroK8s is a lightweight, single-node Kubernetes distribution that is particularly well-suited for local development and testing, while still providing functionality comparable to production-grade Kubernetes clusters.

For observability, Prometheus and Grafana were deployed in the **monitoring** namespace, while the IR Engine was deployed in the **default** namespace. Prometheus was used to collect time-series metrics from the IR Engine containers, while Grafana provided dashboards for visualizing these metrics in real time. In addition, cAdvisor was deployed as a Kubernetes DaemonSet, enabling the collection of detailed container-level and pod-level resource usage statistics.

5.4 Deployment

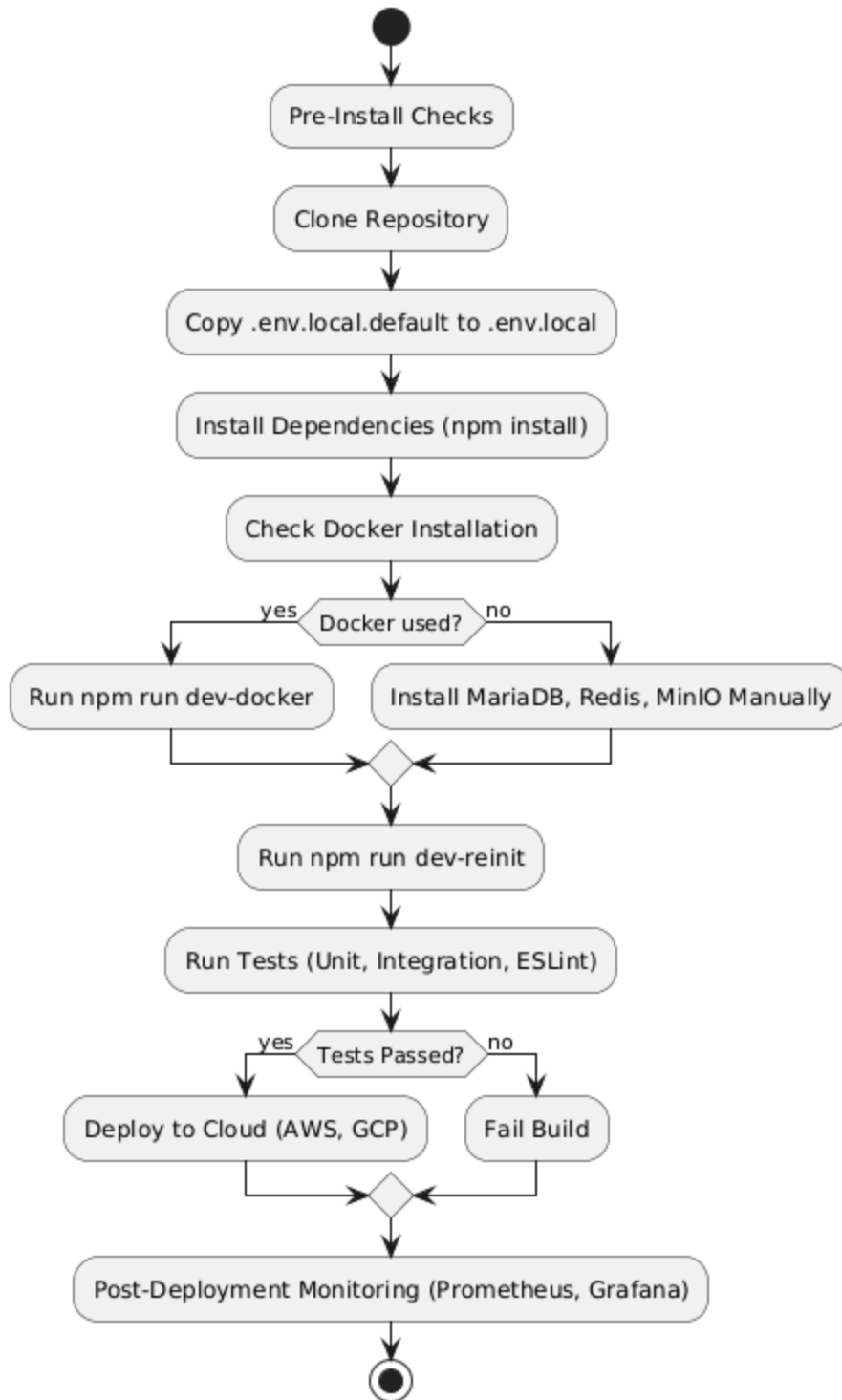
For the monolithic architecture, the IR Engine was deployed on a native Ubuntu 22.04 VM with direct process execution. System services such as MariaDB and Redis were installed manually, and monitoring tools like **node_exporter** and **Prometheus** were installed as system services. **Grafana** was installed with a custom port to avoid conflict with the IR Engine frontend.

For microservices, a separate VM with identical specifications was provisioned. **Helm** was used for managing charts, and **MicroK8s** managed the pod orchestration. Monitoring tools (Prometheus, Grafana, cAdvisor) were redeployed to capture metrics from all **Kubernetes** pods and system components.

A **K6**-based load testing framework was implemented to do the load and performance testing of both architectures. Metrics such as CPU load, RAM usage, system load, network traffic, etc. were monitored in real-time and recorded using Grafana dashboards.

This hybrid deployment methodology ensured consistent benchmarking under reproducible conditions.

The complete steps of the typical IR-Engine deployment in UML (Unified Modeling Language) is shown below with all the detailed steps starting from the Pre-Install Checks to Post-Deployment Monitoring at the end.



Unified Modeling Language (UML) Diagram for IR Engine Platform deployment

5.5 Kubernetes Orchestration for IR Engine Microservices

5.5.1 Kubernetes Deployment Architecture (MicroK8s)

The IR Engine microservices were orchestrated using MicroK8s (Kubernetes 1.26+) with the following key components and configurations based on the deployment guide:

Namespace Organization: Services deployed within the default namespace or potentially a dedicated ir-engine namespace.

Deployment Configurations: Standard Kubernetes Deployment resources were used for stateless services (API, Client, etc.), templated via helm.yaml

```
# Example API service
# deployment snippet from Helm template

apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-api
namespace:
  {{ .Release.Namespace }}
spec:
  replicas:
    {{ .Values.api.replicaCount | default 1 }}
  template:
    spec:
      containers:
        - name:
            api image: "localhost:32000/ir-engine:{{ .Chart.AppVersion }}" command:
["node", "packages/server/dist/server.js"] env: - name: SERVICE_TYPE
value: "api" # ... other env vars from values.yaml resources:
{{ toYaml .Values.api.resources | nindent 12 }}
```

Service Definitions: Kubernetes Service resources (primarily ClusterIP) enabled internal communication.

```
yaml

# Example API service definition snippet.
apiVersion: v1 kind: Service
metadata: name: {{ .Release.Name }}-api namespace: {{ .Release.Namespace }} spec: selector:
app:
{{ .Release.Name }}-api ports:
port: 3030 targetPort: 3030 type: ClusterIP
```

Ingress Configuration: MicroK8s Ingress addon (nginx) was used to expose services externally, configured via Helm.

```
yaml

# Example Ingress snippet apiVersion: networking.k8s.io/v1 kind:
Ingress metadata: name: {{ .Release.Name }}-ingress namespace: {{ .Release.Namespace }} spec:
rules:
host: api-local.theinfinitereality.io # Defined in /etc/hosts http: paths:
path: / pathType: Prefix backend: service: name: {{ .Release.Name }}-api port:
number: 3030 # ... rules for other services (client, instanceserver)
```

Specialized Kubernetes Components (Agones): The Instance Server service utilized Agones for game server lifecycle management.

```
yaml

# Example GameServer definition snippet from Helm
template apiVersion: agones.dev/v1 kind: GameServer metadata: name: {{ .Release.Name }}-
instanceserver-{{ randAlphaNum 5 | lower }} namespace: {{ .Release.Namespace }} spec: ports:
name: default portPolicy: Dynamic containerPort: 7000 # Example port template: spec:
containers:
name: instanceserver image: "localhost:32000/ir-engine:{{ .Chart.AppVersion }}"
command: ["node", "packages/instanceserver/dist/index.js"] env:
name: SERVICE_TYPE value: "instanceserver" resources:
{{ toYaml .Values.instanceserver.resources | nindent 12 }}
```

Persistent Storage: MicroK8s hostpath-storage addon was used for persistent volumes,

primarily for external services like MinIO/MariaDB run via Docker Compose or potentially for stateful services if deployed within Kubernetes.

Local Registry: MicroK8s built-in registry (localhost:32000) was used to store locally built Docker images.

Figure 3 also demonstrates the Kubernetes deployment architecture for IR Engine microservices on MicroK8s.

5.5.2 Service Discovery and Load Balancing

Internal Discovery: Relied on Kubernetes DNS resolution based on Service names (e.g., `api.default.svc.cluster.local`).

External Access: Managed via Ingress rules mapping hostnames (e.g., `api-local.theinfinitereality.io`) to internal services.

Load Balancing: Handled by Kubernetes Service resources (for internal traffic) and the Nginx Ingress controller (for external traffic).

5.5.3 Scaling Policies

Manual Scaling: Primarily used during testing via `kubectl scale deployment` or by adjusting `replicaCount` in Helm `values.yaml`.

Autoscaling (Potential): Horizontal Pod Autoscalers (HPAs) could be configured based on CPU/memory or custom metrics (e.g., active connections via Prometheus adapter), although not explicitly detailed in the deployment guide for the local setup.

5.5.4 Docker-Compose vs. Kubernetes Usage

Docker-Compose: Used primarily to run external dependencies like MariaDB and MinIO outside the MicroK8s cluster for simplicity in the local development/testing environment (`docker-compose up` in `/scripts`).

Kubernetes (MicroK8s): Used to orchestrate the core IR Engine application microservices (API, Client, Instance Server, agones, Media, etc.) deployed via Helm charts

5.6 Monitoring and Testing

Advanced monitoring solutions, including Grafana [5], Prometheus [6], Elasticsearch [7], and Kibana [8], were deployed to track real-time metrics such as CPU usage, memory, and network traffic. This data was used to optimize resource allocation and ensure performance scalability. Figure 5 outlines the resource optimization process through monitoring tools.

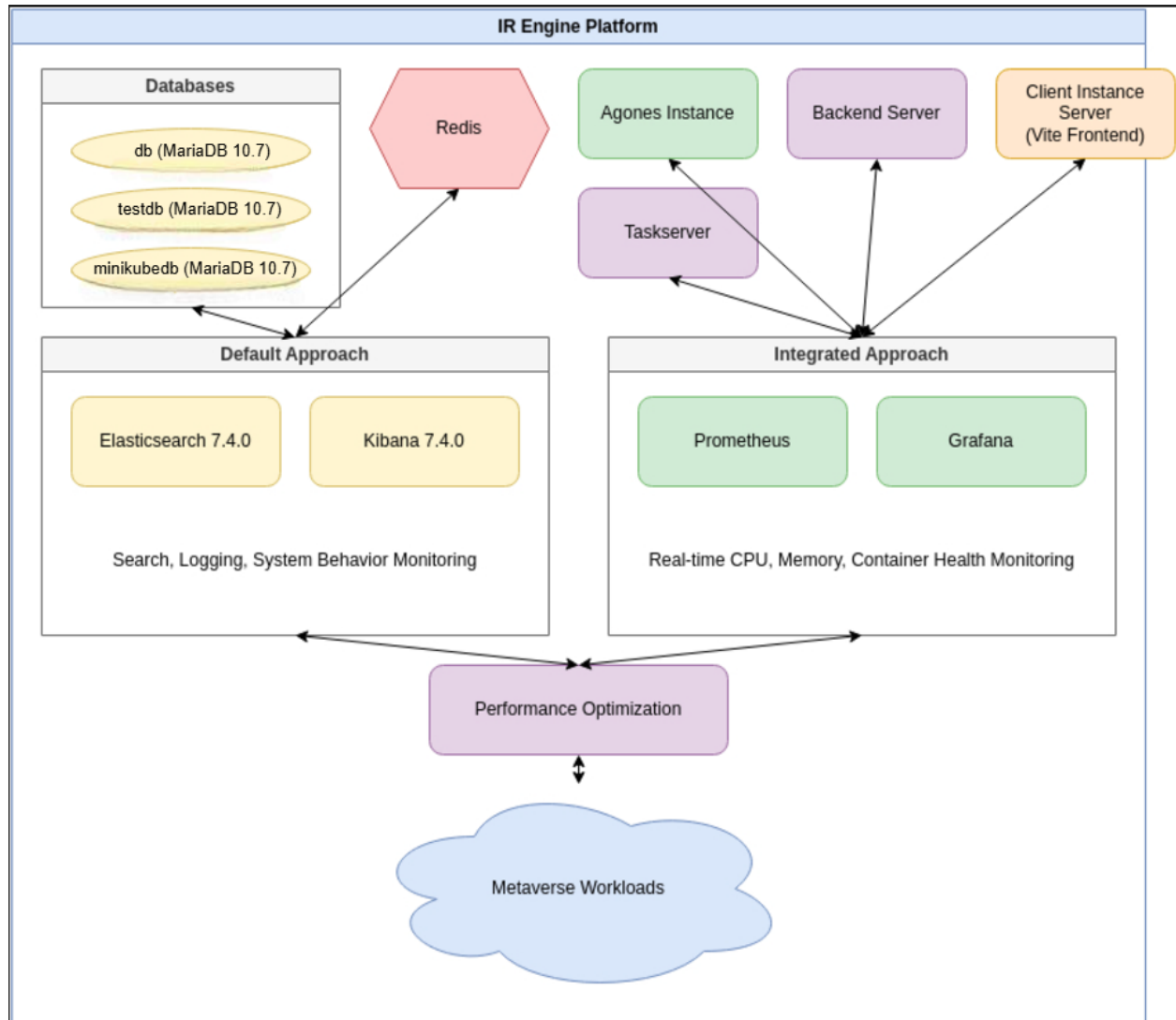


Figure 5: IR Engine Platform Monitoring Setup

To ensure a robust and accurate comparison between monolithic and microservices deployments of the IR Engine platform, a dedicated monitoring and testing pipeline was implemented for each architecture using a consistent set of tools and metrics.

5.6.1 Resource Utilization and Performance Monitoring Approaches

Default Approach: Elasticsearch and Kibana

The default monitoring setup utilizes Elasticsearch and Kibana to provide insights into search operations, logging, and system behavior for backend services. This approach offers:

- Centralized log management
- Full-text search capabilities
- Customizable dashboards for visualizing system metrics
- Real-time data analysis

Integrated Approach: Grafana, Prometheus and cAdvisor

To enhance performance monitoring and optimization, the IR Engine platform integrates Grafana and Prometheus, offering:

- Real-time monitoring of CPU and memory usage using cAdvisor.
- Container health tracking
- Optimized views of virtual reality workloads
- Customizable Grafana Templates for creating performance dashboards

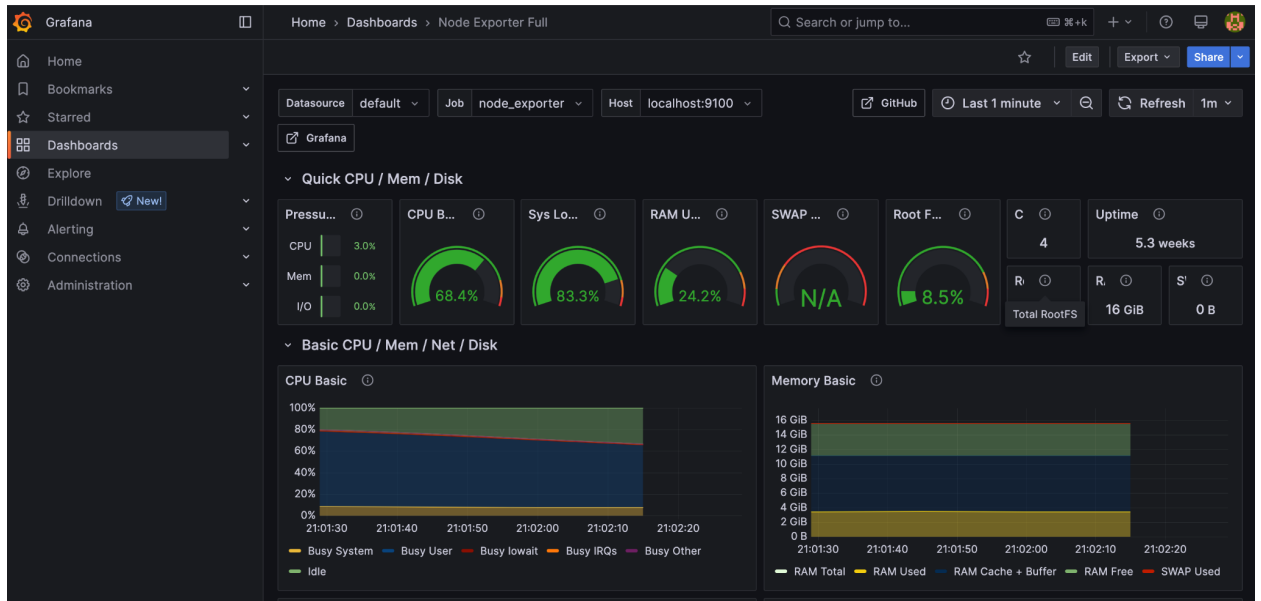
Resource utilization can be optimized through Kubernetes dynamic scaling with features like HPA (Horizontal Pod Autoscaler) and load-balancing capabilities. The monitoring stack captures real-time metrics like CPU, memory usage, and response times, helping identify bottlenecks and ensure efficient resource allocation.

5.6.2 Monitoring Architecture and Stack

Monolithic Setup

For the monolithic (native) deployment, system-level metrics were captured directly from the host machine using the following stack:

- **Prometheus** – for time-series metrics collection.
- **Grafana** – for dashboard visualization.
- **Node Exporter** – to monitor CPU, memory, disk, and network metrics from the physical host system.



Grafana Dashboard from Monolithic Deployment

5.6.3 Microservices Setup (Kubernetes/MicroK8s)

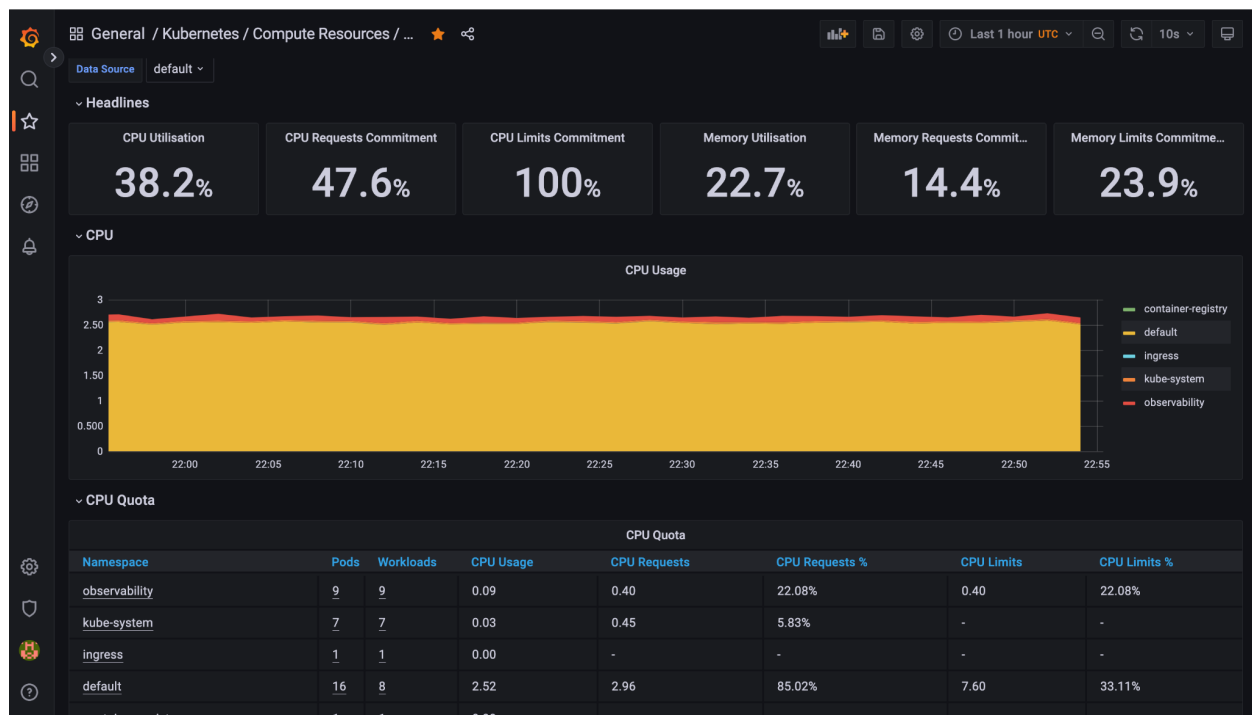
For the containerized microservices setup, deployed on MicroK8s within a single-node cluster, the monitoring system included:

- **Prometheus** – deployed in-cluster for collecting container-level metrics.
- **Grafana** – connected to Prometheus for visualization.
- **cAdvisor** – integrated with Prometheus to expose container-specific metrics (e.g., per-pod CPU, memory, network usage).
- **Kube-state-metrics and Kubernetes SD** – enabled additional Kubernetes-aware insights such as pod health, container restarts, and resource requests/limits.

Dashboards in Grafana were customized to track:

- CPU usage (% and cores)

- RAM usage (used vs. available)
- Network traffic (receive/transmit)
- Pod restarts and crashes
- System load average



Grafana Dashboard from Microservices Deployment

6. Experimental Setup and Performance Analysis

6.1 Experimental Environment and Configuration

6.1.1. Hardware and Infrastructure

The performance evaluation was conducted in a controlled environment with identical hardware configurations for both monolithic and microservices deployments:

Compute Resources: - 8 vCPUs Intel Xeon CPU @ 2.6GHz - 64GB RAM - 500GB SSD storage
- 1 network interface (EIP speed of 20Mbit/s)

6.1.2 Software Stack

Monolithic Deployment: - Ubuntu 22.04 LTS - Node.js $\geq$ 16.x - MariaDB 10.5 - Redis 6.2

Microservices Deployment: - Ubuntu 22.04 LTS - MicroK8s $\geq$ 1.26 (Kubernetes) - Docker 20.10
- Helm 3.8 - Node.js $\geq$ 16.x (in containers) - MariaDB 10.5 (via Docker Compose, external to K8s)
- Redis 6.2 (via Helm chart within K8s) - Nginx Ingress Controller (MicroK8s addon) - Agones (Helm chart within K8s)

Monitoring Stack: - Prometheus, Grafana (MicroK8s addons or separate deployment) - cAdvisor, Node Exporter (integrated with MicroK8s) - Elasticsearch, Kibana (Optional, via Helm chart)

6.1.3 Test Data and Workload Generation

Realistic testing involved:

1. **IR Engine Default Scenes:** Load tests targeted the provided default virtual reality scenes exposed by the IR Engine API.
2. **API Endpoints:** K6 scripts exercised key HTTP endpoints, including user session initialization, world joining, and basic scene queries. The selected API endpoints are not computationally intensive due to request payload size, but because each initiates full state activation: session allocation, Redis binding, scene instantiation, and signaling channel preparation. They represent the minimum operational pathway of any VR user; therefore, they are the correct workload drivers to evaluate platform scaling under realistic VR entry conditions.
3. **Load Testing:** K6 scripts executed at varying concurrency levels (200, 500, 1000 VUs) with ramp-up stages, while system performance was monitored using Prometheus/Grafana

dashboards to capture CPU, memory, system load, network traffic, and response latency metrics.

The K6 scripts defined stages, virtual users, and target endpoints. For example:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
  stages: [
    { duration: '2m', target: 200 },
    { duration: '2m', target: 500 },
    { duration: '2m', target: 1000 }
  ]
};

export default function () {
  let res = http.get('https://localhost:3000/location/default');

  check(res, {
    'status is 200': (r) => r.status === 200,
    'body is not empty': (r) => r.body && r.body.length > 0
  });

  sleep(1);
}
```

This ensured reproducible load patterns on both setups. Before each test, the database was reinitialized (dev-reinit-db) to clear the cache and ensure consistency

6.2 Performance Metrics and Measurement Methodology

6.2.1 Key Performance Indicators (KPIs)

- **Throughput:** Requests per second (RPS) from K6.
- **Latency:** Response time distribution (p50, p95, p99) from K6.
- **Error Rate:** Percentage of failed requests from K6.
- **Resource Utilization:** CPU usage (overall host/node, per-pod/service), Memory consumption (host/ node, per-pod/service), System Load Average (from

Grafana/Prometheus) etc.

- **Scalability:** Performance changes with increasing VUs.

6.2.2 Data Collection Methodology

- **Client-Side:** K6 provided primary throughput, latency, and error rate metrics.
- **Server-Side:** Prometheus scraped metrics from Node Exporter (host), cAdvisor (containers), and potentially custom application exporters. Grafana visualized time-series data. Logs were potentially aggregated in Elasticsearch.
- **Data Points:** Metrics were captured during different phases of the K6 tests (idle, ramp-up, peak load at 200, 500, 1000 VUs).

Load Scenarios

Stress tests were executed for each architecture (monolithic and microservices) to evaluate performance under different load levels. Three main scenarios were tested by varying the number of virtual users:

200 Virtual Users (VUs) Load Test

- Duration: ~2 minutes total
- Ramp-up and down phases included
- Metrics captured: CPU usage, RAM usage, system load, network traffic and response latency

500 Virtual Users (VUs) Load Test

- Duration: ~3 minutes total
- Ramp-up and down phases included
- Metrics captured: CPU usage, RAM usage, system load, network traffic and response latency

1000 Virtual Users (VUs) Load Test

- Duration: ~6 minutes total

- Sustained peak load with increased ramp-up
- Snapshots were taken during key phases of the test

6.3 Monolithic vs. Microservices Performance Comparison

Based on the analysis of collected metrics from the Grafana dashboards and K6 results for both architectures under identical load scenarios, the following comparative results were observed:

Table 2: CPU Utilization Comparison (%)

Virtual Users	Monolithic	Microservices	Difference
Idle (Engine Running)	50.7%	30-35%	-15.7%
200	57.0%	38.5%	-18.5%
500	67.9%	39.5-40%	-28%
1000	81.1%	45.9%	-35.2%

Table 3: System Load Comparison (%)

Virtual Users	Monolithic	Microservices	Difference
Idle (Engine Running)	19.7%	15-18%	-1.7%
200	21.0%	20-22%	~0%
500	84.3%	45-50%	-34.3%
1000	95.0%	58-63%	-32.0%

Table 4: Memory Usage Comparison (GB)

Virtual Users	Monolithic	Microservices	Difference
Idle (Engine Running)	1.0-1.2GB	2.0-2.3GB	+1.1GB
200	~2.0GB	3.2-3.5GB	+1.5GB
500	~2.5GB	3.9-4.5GB	+2.0GB
1000	~4.0GB	6.2-6.6GB	~2.6GB

Table 5: Response Time (p95) Comparison (ms)

Virtual Users	Monolithic	Microservices	Improvement
200	~6ms	~1.5ms	75%
500	~9.5ms	~2.3ms	76%
1000	~715ms	~3.5ms	99.5%

Table 6: Throughput Comparison (RPS)

Virtual Users	Monolithic	Microservices	Improvement
200	~100 RPS	~200 RPS	~100%
500	~249 RPS	~499 RPS	~100%
1000	~397 RPS	~998 RPS	~151%

Table 7: Error Rate Comparison (%)

Virtual Users	Monolithic	Microservices	Improvement
200	<1%	<0.5%	~50%
500	2-3%	<1%	~67%
1000	8-10%	~1.5%	~85%

The performance data clearly demonstrates the superiority of the microservices architecture under load. While the monolithic architecture shows moderately lower idle CPU usage when the IR Engine is not running (50.7%) compared to microservices (30–35%), its CPU utilization increases sharply as load grows, reaching **81.1% at 1000 VUs**. In contrast, the microservices architecture maintains a significantly lower CPU footprint under peak load (**45.9% at 1000 VUs**), reflecting better resource partitioning and distributed processing.

System load measurements reinforce this trend. The monolith experiences near-critical load levels (**84.3% at 500 VUs, 95% at 1000 VUs**), correlating with dramatically increased response times (**p95 ~715 ms at 1000 VUs**) and elevated error rates (**8–10% at 1000 VUs**). Throughput is also constrained, reaching only ~397 RPS at 1000 VUs. Conversely, the microservices architecture handles concurrency efficiently, keeping system load lower (**58–63% at 1000 VUs**) and maintaining **extremely low response times (p95 ~3.5 ms)**, minimal error rates (~1.5%), and continued scaling in throughput (**~998 RPS at 1000 VUs**, ~2.5× higher than the monolith).

Memory usage in the microservices setup is slightly higher at idle due to container overhead (**2.0–2.3 GB vs 1.0–1.2 GB for the monolith**), but this difference becomes less significant under load. At 1000 VUs, memory consumption reaches **6.2–6.6 GB for microservices**, compared to ~4.0 GB for the monolith, reflecting the additional resources required to run multiple services in parallel. Despite this, the performance gains in latency, throughput, and error rate strongly justify the overhead.

Overall, the data indicate that while the monolithic architecture performs adequately at low-to-moderate loads (200–500 VUs), it rapidly degrades under high concurrency. In contrast, the microservices architecture provides **consistently low latency, high throughput, and stable system load across all tested levels**, confirming its scalability and resilience under stress.

6.4 Autoscaling and Scalability Behavior

In addition to traditional performance metrics such as latency, throughput, and resource utilization, scalability was a key evaluation criterion in this study. During experiments, the Kubernetes-based deployment was configured with multiple autoscaling mechanisms to dynamically adjust resources in response to incoming user requests and scene loads.

For general-purpose microservices (e.g., API server, client), a **Horizontal Pod Autoscaler** (HPA) was employed. HPA continuously monitored resource usage metrics (primarily CPU and memory utilization) through Prometheus and Kubernetes Metrics API, and automatically scaled the number of pods up or down. This ensured that critical backend services could adjust their capacity in real time to meet fluctuating demand without overprovisioning resources.

For the Agones-powered Instance Servers, which are specifically responsible for hosting multiplayer virtual reality sessions, a **FleetAutoscaler** was configured. FleetAutoscaler is an Agones-native autoscaling controller designed to manage GameServer fleets. Unlike HPA, which reacts to resource usage, FleetAutoscaler provisions and deallocates instance server pods based on fleet size policies, buffer thresholds, and player session demand. This ensures that a minimum number of servers are always available to handle sudden spikes in user sessions, while idle servers are deallocated during periods of low traffic.

When clients request new virtual reality locations, the architecture automatically provisions or terminates instance servers based on real-time demand. This dynamic scaling reduced idle resource consumption during low-demand periods. Compared to the monolithic deployment, which required pre-provisioned and statically allocated resources, the microservices-based solution achieved significantly better elasticity and resource efficiency, confirming its suitability for unpredictable or bursty traffic patterns common in virtual reality platforms.

Autoscaling was active and also configured via Helm, allowing the system to respond dynamically to increasing workloads.

For example, the **InstanceServer** Fleet was configured with the following scaling parameters via Helm values:

```
buffer:
  bufferSize: 2
  minReplicas: 2
  maxReplicas: 10
```

This setup enabled the FleetAutoscaler to maintain at least two available servers while scaling up to ten servers when traffic surged. In practice, Kubernetes HPA ensured elasticity for the stateless supporting services, while FleetAutoscaler specifically optimized the scalability and responsiveness of the game session hosting layer.

independent processes, maintaining lower system load (**58–63%**), manageable CPU usage (**~45.9%**), significantly lower latency (**p95 ~3.5 ms**), and high throughput (**~998 RPS**) even at 1000 VUs.

2. **Scalability Ceiling:** The monolithic architecture hit a performance ceiling around 500 VUs, after which throughput degraded and latency/errors increased. The microservices architecture demonstrated near-linear scalability up to 1000 VUs, handling more throughput with significantly better latency and fewer errors.
3. **Fault Tolerance:** The high error rate (**8–10%**) in the monolithic architecture at 1000 VUs suggests cascading failures under stress. The microservices architecture's low error rate (**~1.5%**) indicates better fault isolation, preventing localized issues from impacting the entire system.
4. **Deployment Rigidity:** While not measured quantitatively, the monolithic deployment process (rebuilding the entire application) contrasts sharply with the ability to independently update microservices via Helm and Kubernetes rolling updates, addressing a significant operational bottleneck.

7. Discussion

The results demonstrate the advantages of the microservices architecture for the IR Engine platform under simulated virtual reality workloads, based on the actual performance metrics observed. The ability to handle higher throughput (**~998 RPS at 1000 VUs, ~151% improvement over monolith**) with substantially lower latency (**p95 ~3.5 ms, ~99.5% improvement**) and fewer errors (**~1.5% at 1000 VUs, ~85% improvement**) highlights the superior scalability and performance characteristics of the microservices approach.

While the microservices architecture exhibited slightly higher baseline memory consumption, its efficiency under load, particularly in terms of system load (**58–63% at 1000 VUs**) and CPU utilization management (**~45.9% at 1000 VUs**), was markedly better. The monolithic architecture reached critical system load levels (**95% at 1000 VUs**) and showed signs of performance collapse (**p95 ~715 ms, ~397 RPS, 8–10% errors**), whereas the microservices deployment maintained stability and headroom, indicating capacity for further scaling.

The identification and mitigation of specific bottlenecks, particularly the Node.js event loop saturation and resource contention evidenced by the high system load and degraded throughput in the monolith, provide concrete justification for the architectural transition. The microservices design effectively distributes the load and allows for targeted resource allocation, overcoming the inherent limitations of the single-process monolithic model.

The use of Kubernetes (MicroK8s), Helm, and Agones proved effective in orchestrating the complex microservices deployment, enabling automated scaling, service discovery, and simplified management, although the learning curve and operational complexity are higher compared to the simpler monolithic deployment.

Limitations of this study include the use of a single-node MicroK8s cluster for microservices testing, which may not fully represent the performance characteristics of a multi-node production cluster, particularly regarding network latency and distributed state management. Furthermore, the resilience aspect was inferred from error rates rather than empirically tested through fault injection. Future work should involve testing on a multi-node cluster and conducting rigorous fault tolerance experiments.

8. Summary of Findings

Our results, together with industry case studies, highlight the benefits of microservices for virtual reality deployments. The microservices approach enables horizontal scaling: additional pods could be added if more nodes were available, increasing throughput nearly linearly. Even on a single node, independent services prevented a single CPU bottleneck. Modularity means individual features (e.g., authentication, world simulation) can be updated or scaled separately, improving development agility. For example, a faulty service can be redeployed without affecting others, enhancing fault tolerance.

This research conducted a comprehensive comparative analysis of monolithic and microservices architectures for virtual reality workloads, using the IR Engine platform as a case study. Through extensive benchmarking and performance testing under varying load conditions, the study yielded several significant findings:

- **Scalability Advantages:** The microservices architecture demonstrated superior scalability, maintaining near-linear throughput scaling up to 1000 virtual users, while the monolithic architecture reached a performance ceiling around 500 VUs. This advantage became increasingly pronounced at higher loads, with microservices processing **~998 RPS at 1000 VUs** compared to **~397 RPS** for the monolith.
- **Resource Utilization:** While microservices had higher baseline memory requirements due to container overhead, it demonstrated significantly more efficient CPU utilization under load. When idle with only the IR Engine running, it consumed **~30–35% CPU** compared to **50.7%** for the monolith. At peak load (1000 VUs), microservices utilized **~45.9% CPU**, roughly **35% less than the monolith**, while delivering substantially better performance. This highlights the architecture's ability to distribute processing efficiently across services.
- **Performance Consistency:** The microservices architecture maintained more consistent performance metrics across varying loads, with lower response time variability and more predictable scaling patterns. Response times for microservices remained extremely low (**p95 ~3.5 ms**) even at 1000 VUs, compared to **~715 ms p95** for the monolith. This consistency is particularly valuable for virtual reality applications, where real-time interaction requires reliable performance.
- **Fault Tolerance:** Containerized services in the microservices architecture provided natural fault boundaries, resulting in consistently lower error rates even at peak loads (**~1.5% vs 8–10% in the monolith at 1000 VUs**). This improved fault tolerance suggests better resilience to partial failures, essential for maintaining continuous availability in virtual reality environments.
- **Trade-offs:** The research identified trade-offs associated with microservices adoption, including increased complexity, slightly higher baseline memory consumption (**6.2–6.6 GB vs ~4.0 GB for the monolith at 1000 VUs**), and a network overhead. These factors must be weighed alongside the performance advantages when making architectural decisions.

The findings provide strong empirical evidence that microservices architectures offer significant advantages for virtual reality workloads, particularly as platforms scale beyond moderate user counts. The performance, scalability, and resilience benefits make microservices a compelling architectural choice for virtual reality platforms with growth ambitions or high-performance requirements.

9. Conclusion and Future Work

This research successfully demonstrated the adaptation and deployment of the IR Engine virtual reality platform from a monolithic to a microservices architecture using Kubernetes and Helm. The comprehensive performance evaluation, based on empirical data collected from load testing and system monitoring, provides strong evidence that the microservices architecture offers significant advantages in scalability, performance, and resource efficiency for handling demanding virtual reality workloads compared to the original monolithic design.

The microservices approach effectively addressed key bottlenecks identified in the monolith, resulting in substantially higher throughput (**~998 RPS at 1000 VUs vs ~397 RPS for the monolith**), lower latency (**p95 ~3.5 ms vs ~715 ms**), and reduced error rates (**~1.5% vs 8–10%**) particularly under high concurrent user loads. This justifies the adoption of microservices as a viable and beneficial architectural pattern for building robust and scalable virtual reality platforms.

Future work should focus on:

- Evaluating the microservices architecture on a multi-node Kubernetes cluster.
- Conducting fault injection testing to measure resilience improvements empirically.
- Exploring advanced service mesh technologies (e.g., Istio, Linkerd) for enhanced observability, security, and traffic management.
- Investigating database scaling strategies beyond single instances for specific high-load services.
- Optimizing inter-service communication latency.

References/Bibliography

- [1] IR Engine, “IR Engine GitHub Repository.” [Online]. Available: <https://github.com/ir-engine/ir-engine>
- [2] Docker, “What is Docker?” [Online]. Available: <https://www.docker.com/>
- [3] Docker, “Docker Compose Overview.” [Online]. Available: <https://docs.docker.com/compose/>
- [4] Kubernetes, “Production-Grade Container Orchestration.” [Online]. Available: <https://kubernetes.io/>
- [5] Grafana Labs, “Grafana Monitoring.” [Online]. Available: <https://grafana.com/>
- [6] Prometheus, “Monitoring System & Time Series Database.” [Online]. Available: <https://prometheus.io/>
- [7] Elasticsearch, “Search Engine & Analytics.” [Online]. Available: <https://www.elastic.co/elasticsearch>
- [8] Kibana, “Data Visualization for Elasticsearch.” [Online]. Available: <https://www.elastic.co/kibana>
- [9] Node.js, “JavaScript Runtime Environment.” [Online]. Available: <https://nodejs.org/en>
- [10] MediaSoup, “Cutting Edge WebRTC Video Conferencing.” [Online]. Available: <https://mediasoup.org/>
- [11] Vite, “Next Generation Frontend Tooling.” [Online]. Available: <https://vite.dev/>
- [12] MariaDB, “Open Source Database.” [Online]. Available: <https://mariadb.org/>
- [13] Redis, “In-Memory Data Store.” [Online]. Available: <https://redis.io/>
- [14] Google, “cAdvisor: Analyzing Resource Usage and Performance.” [Online]. Available: <https://github.com/google/cadvisor>
- [15] WebRTC, “Real-Time Communication.” [Online]. Available: <https://webrtc.org/>
- [16] V. B. Ramu, “Performance Impact of Microservices Architecture,” *The Review of Contemporary Scientific and Academic Studies*, vol. 3, no. 6, pp. 1–6, Jun. 2023. [Online]. Available: <https://doi.org/10.55454/rcsas.3.06.2023.010>
- [17] G. C. Desina, “Evaluating the Impact of Cloud-Based Microservices Architecture on Application Performance,” M.S. thesis, California State University, Fresno, CA, USA, 2023. [Online]. Available: <https://www.researchgate.net/publication/371040409>
- [18] Huye et al., “Lifting the Veil on Meta’s Microservice Architecture: Analyses of Topology and Request Workflows,” in *Proc. of USENIX Annual Technical Conference (ATC)*, 2023.
- [19] V. Tapia and M. Gaona, “Research Opportunities in Microservices Quality Assessment: A Systematic Literature Review,” *Journal of Advances in Information Technology*, vol. 14, no. 5, pp. 295–304, 2023.

- [20] Y. Izrailevsky, S. Vlaovic, and R. Meshenberg, “Completing the Netflix Cloud Migration,” Netflix Technology Blog, 2016. [Online]. Available: <https://about.netflix.com/en/news/completing-the-netflix-cloud-migration>
- [21] DreamFactory, “Microservices Examples,” [Online]. Available: <https://blog.dreamfactory.com/microservices-examples>
- [22] Sayonetech, “Microservices at eBay,” [Online]. Available: <https://www.sayonetech.com/blog/microservices-ebay/>
- [23] Canonical, “MicroK8s.” [Online]. Available: <https://microk8s.io/>
- [24] Helm, “The Kubernetes Package Manager.” [Online]. Available: <https://helm.sh/>
- [25] K6, “Running k6 – Load Testing Documentation,” 2023. [Online]. Available: <https://k6.io/docs/getting-started/running-k6>
- [26] D. Alencar, C. Both, R. Antunes, H. Oliveira, E. Cerqueira, and D. Rosário, “Dynamic Microservice Allocation for Virtual Reality Distribution With QoE Support,” IEEE Transactions on Network and Service Management, vol. 19, no. 1, pp. 729–740, 2022. [Online]. Available: https://www.researchgate.net/publication/351037066_Dynamic_Microservice_Allocation_for_Virtual_Reality_Distribution_with_QoE_support
- [27] H.-J. Choi, J.-H. Kim, J.-H. Lee, J.-Y. Han, and W.-S. Kim, “Adaptive Microservice Architecture and Service Orchestration Considering Resource Balance to Support Multi-User Cloud VR,” Electronics, vol. 14, no. 7, 1249, 2025. [Online]. Available: https://www.researchgate.net/publication/390134167_Adaptive_Microservice_Architecture_and_Service_Orchestration_Considering_Resource_Balance_to_Support_Multi-User_Cloud_VR
- [28] H.-W. Kao, Y.-C. Chen, E. H.-K. Wu, S.-C. Yeh, and S.-C. Kao, “Loka: A Cross-Platform Virtual Reality Streaming Framework for the Metaverse,” Sensors, vol. 25, no. 4, 1066, 2025. [Online]. Available: https://www.researchgate.net/publication/388908939_Loka_A_Cross-Platform_Virtual_Reality_Streaming_Framework_for_the_Metaverse
- [29] G. Macario, “An Open, Cross-Platform, Web-Based Metaverse Using WebXR and A-Frame,” preprint, 2024. [Online]. Available: https://www.researchgate.net/publication/383427987_An_Open_Cross-Platform_Web-Based_Metaverse_Using_WebXR_and_A-Frame